Doc-Genie

Professional Python Function Docstring Generator with AST Analysis

Project Description

Doc-Genie - is an intelligent Python documentation platform that leverages Abstract Syntax Tree (AST) analysis and advanced code understanding to provide comprehensive automatic docstring generation with professional styling through Gradio's interactive interface. The platform addresses the challenge of maintaining consistent and accurate Python documentation by delivering AI-powered function analysis, automatic docstring generation, multiple documentation styles (Google/NumPy), intelligent logic detection, and production-ready export capabilities.

Using sophisticated AST parsing algorithms, Python's introspection capabilities, and deep code analysis, the system analyzes Python function definitions, extracts function signatures and parameters, performs logic analysis to understand actual behavior, generates contextually accurate docstrings in multiple styles, and provides comprehensive export options (PDF, TXT) with social media sharing capabilities. The system ensures professional documentation through Gradio's user-friendly interface while maintaining accuracy through intelligent code analysis and pattern recognition.

Doc-Genie - transforms Python documentation from a tedious manual task into an intelligent, user-friendly experience through its modern Gradio interface, comprehensive feature set, and advanced AST analysis that understands function behavior, maintains consistency across projects, and generates production-ready documentation that meets industry standards.

Scenarios

Scenario 1: Legacy Code Documentation

A software engineer inherits a large codebase with hundreds of functions lacking proper documentation. Using Doc-Genie, they upload Python files and quickly generate comprehensive Google-style docstrings for all functions. The tool's AST analysis automatically detects parameter types, return values, and function behavior, creating accurate documentation without manual inspection. This significantly accelerates the documentation process and ensures consistency across the entire codebase.

Scenario 2: API Development & Documentation

A backend developer building REST APIs needs to maintain consistent documentation as they create new endpoints. Doc-Genie analyzes each function and generates NumPy-style docstrings matching their team's standards. The tool correctly identifies parameters, their types, and return values from function signatures, then performs logic analysis to create contextual descriptions of what each function does. This ensures that API documentation is always synchronized with actual implementation.

Scenario 3: Team Collaboration & Standards

A team of developers needs to standardize documentation across multiple projects. Doc-Genie allows them to enforce consistent docstring formats across the codebase. The PDF export feature generates comprehensive documentation reports that can be reviewed and archived, while the history tracking ensures nothing is lost. Team members can share their documented code via WhatsApp or Facebook, facilitating code review and collaboration.

Scenario 4: Educational & Training Purpose

An instructor teaching Python programming uses Doc-Genie to demonstrate best practices in documentation. Students write functions, and the tool generates professional docstrings, showing them the expected standard. By analyzing student code, it provides feedback on function clarity and suggests improvements. The tool helps students understand how to write self-documenting code and follow industry conventions.

Scenario 5: Code Review & Quality Assurance

A QA team reviewing code submissions uses Doc-Genie to quickly check if functions have adequate documentation. The tool generates expected docstrings for each function, and reviewers can compare generated documentation with existing documentation to ensure completeness and accuracy. This prevents undocumented or poorly documented functions from reaching production, maintaining code quality standards.

Architecture Overview

Doc-Genie - is built as a modular platform combining Python's AST module with Gradio for an intelligent documentation generation system. The architecture prioritizes code analysis accuracy, documentation quality, and user experience through advanced parsing techniques and comprehensive output options.

Core Technologies

- Python 3.8+: Core language with AST module for code analysis
- Gradio 4.0+: Interactive web interface framework
- AST (Abstract Syntax Tree): Python's built-in code parsing library
- ReportLab: PDF generation for comprehensive documentation reports
- Regular Expressions: Pattern matching for code analysis
- Google Colab: Cloud-based development and deployment environment
- HTML5/CSS3: Modern web standards for interface styling

Pre-requisites

Software Requirements

- Python 3.8+: Download from python.org (AST support required)
- Google Account: Required for Google Colab access
- pip: Python package manager (comes with Python)
- Web Browser: Chrome, Firefox, Safari, or Edge
- Internet Connection: Required for Colab and Gradio sharing

Knowledge Prerequisites

- Python Fundamentals: Functions, decorators, type hints
- Python AST: Understanding of Abstract Syntax Trees and code parsing
- Python Docstrings: Familiarity with Google and NumPy docstring formats
- Gradio Framework: Understanding of interface components and callbacks
- Code Analysis: Basic understanding of static code analysis concepts
- Google Colab: Basic notebook interface and cell execution

Hardware Requirements

- Processor: Any modern processor (Intel i3 or equivalent)
- RAM: Minimum 2GB local (Colab provides cloud resources)
- Storage: 500MB free space for application and generated PDFs
- Display: Minimum 1024x768 resolution for interface usability

Project Workflow

Phase 1: Environment Setup & Core Architecture

Activity 1.1: Install Dependencies & Configure Environment

- Create requirements.txt with all dependencies
- Set up Google Colab environment with proper settings
- Install required packages: gradio, transformers, torch, reportlab, requests
- Verify all packages installed and Python AST module available

Activity 1.2: Design DocGenieAnalyzer Core Class

- Create DocGenieAnalyzer class for code analysis
- Design AST parsing engine for function extraction
- Implement type annotation detection
- Set up generation history tracking system

Phase 2: Advanced Code Analysis Engine

Activity 2.1: Implement Function Signature Extraction

- Create extract_function_signature() method
- Parse function name from AST
- Extract parameter information including names and types
- Detect default parameter values
- Extract return type annotations

Activity 2.2: Build Deep Logic Analysis System

- Create analyze_function_logic() method
- Detect loops (for, while) in function body
- Identify conditional statements (if/elif/else)
- Detect mathematical operations (add, multiply, divide, subtract)
- Track return statements and return types
- Identify function calls and external dependencies

- Generate contextual descriptions from analysis results

Phase 3: Interface Development & User Experience

Activity 3.1: Build Main Code Input & Output Interface

- Create code input textarea with syntax highlighting
- Implement file upload for .py files
- Add docstring style selector (Google/NumPy radio buttons)
- Create code output display with syntax highlighting
- Add generate button with click handler

Activity 3.2: Implement Export Functionality Tabs

- Create PDF export using ReportLab
- Create TXT export with formatted output
- Implement file download capability
- Add export status messages

Phase 4: Export & Reporting System

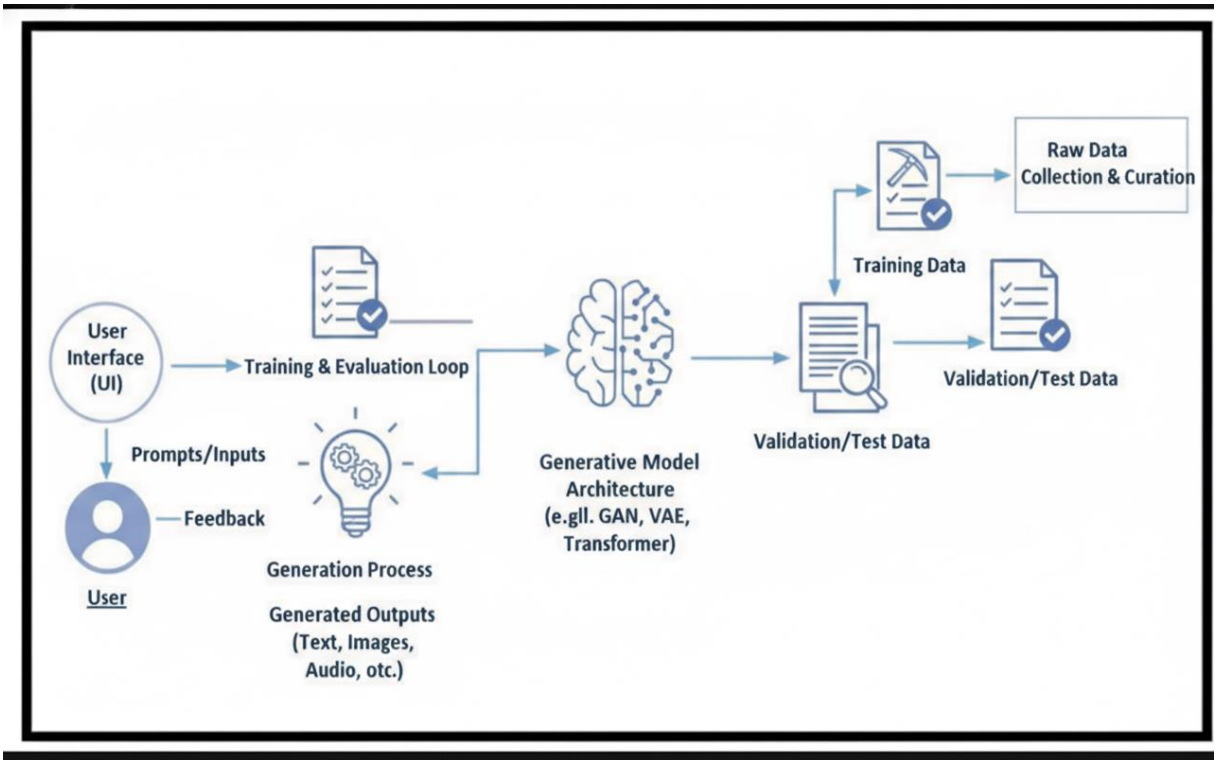
Activity 4.1: Implement PDF Report Generation

- Create ReportLab document structure
- Design professional PDF styling
- Format code blocks with proper syntax
- Add pagination for multiple documentations
- Include metadata and timestamps

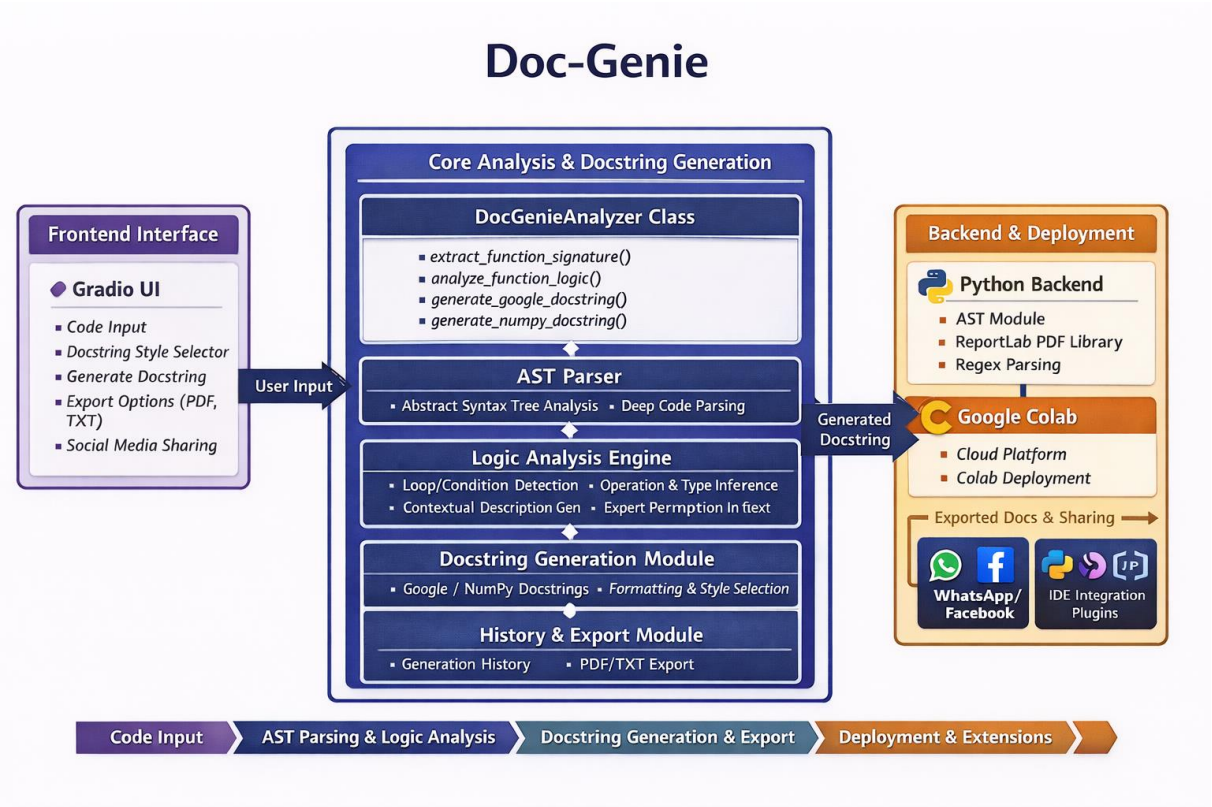
Activity 4.2: Implement Text File Export

- Create formatted text output
- Add clear section separators
- Include metadata and analysis information
- Support batch documentation export

Technical Architecture:



Project Structure



MILESTONE 1: Environment Setup & Core Architecture

Activity 1.1: Install Dependencies

Create requirements.txt with the following dependencies:

- gradio==4.41.0
- transformers==4.35.0
- torch==2.0.0
- reportlab==4.0.7
- requests==2.31.0
- black==24.1.1
- autopep8==2.1.0

Install via: pip install -r requirements.txt

```
[7]
✓ 3s
!pip install -q gradio transformers torch black autopep8 reportlab requests ast-super-parser

import gradio as gr
from datetime import datetime
from pathlib import Path
import json
import re
import ast
from reportlab.lib.pagesizes import letter, A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, PageBreak, Table, TableStyle, Preformatted
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
from reportlab.lib import colors
import requests
import urllib.parse
import inspect
```

Code reference (hint: click on image and you can copy paste from Comment)

Activity 1.2: Initialize Core Analyzer Class

Create DocGenieAnalyzer class with following structure:

```
class DocGenieAnalyzer:
    @staticmethod
    def extract_function_signature(code):
    - Parse AST from code
    - Extract function name, parameters, return type
    - Return signature dictionary

    @staticmethod
    def analyze_function_logic(func_def, code):
    - Walk through AST nodes
    - Detect loops, conditions, operations
    - Generate contextual description
    - Return analysis dictionary
```

@staticmethod
def generate_google_docstring(signature, analysis, code):
- Format Google-style docstring
- Include summary, args, returns
- Return formatted string

@staticmethod
def generate_numpy_docstring(signature, analysis, code):
- Format NumPy-style docstring
- Include summary, parameters, returns
- Return formatted string

```python
class DocGenieAnalyzer:
    """Analyze Python functions and generate professional docstrings"""

    @staticmethod
    def extract_function_signature(code):
        """Extract function name, parameters, return type from code"""
        try:
            tree = ast.parse(code)
            func_def = tree.body[0]
            if not isinstance(func_def, ast.FunctionDef):
                return None, None

            func_name = func_def.name
            params = []

            for arg in func_def.args.args:
                param_type = "Any"
                if arg.annotation:
                    param_type = ast.unparse(arg.annotation) if hasattr(ast, 'unparse') else "Any"
                params.append({
                    'name': arg.arg,
                    'type': param_type,
                    'default': None
                })
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
            return None, None

    @staticmethod
    def analyze_function_logic(func_def, code):
        """Deep analysis of function body to understand actual behavior"""
        try:
            body = func_def.body
            analysis = {
                'has_loops': False,
                'has_conditions': False,
                'has_return': False,
                'loop_types': [],
                'condition_types': [],
                'operations': [],
                'return_types': [],
                'variables_used': set(),
                'function_calls': [],
                'description': ""
            }

            # Walk through AST
            for node in ast.walk(func_def):
                if isinstance(node, ast.For):
                    analysis['has_loops'] = True
                    analysis['loop_types'].append('for')
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
        elif isinstance(node, ast.While):
            analysis['has_loops'] = True
            analysis['loop_types'].append('while')
        elif isinstance(node, ast.If):
            analysis['has_conditions'] = True
            analysis['condition_types'].append('if')
        elif isinstance(node, ast.Return):
            analysis['has_return'] = True
            if node.value:
                analysis['return_types'].append(ast.unparse(node.value) if hasattr(ast, 'unparse') else 'value')
        elif isinstance(node, ast.BinOp):
            op = node.op
            if isinstance(op, ast.Mult):
                analysis['operations'].append('multiplication')
            elif isinstance(op, ast.Add):
                analysis['operations'].append('addition')
            elif isinstance(op, ast.Sub):
                analysis['operations'].append('subtraction')
            elif isinstance(op, ast.Div):
                analysis['operations'].append('division')
        elif isinstance(node, ast.Call):
            if isinstance(node.func, ast.Name):
                analysis['function_calls'].append(node.func.id)
        elif isinstance(node, ast.Name):
            analysis['variables_used'].add(node.id)

    # Build detailed description
    description_parts = []

    if func_def.name:
        description_parts.append(f"Executes {func_def.name} operation")

    if analysis['has_conditions']:
        description_parts.append("with conditional logic")

    if analysis['has_loops']:
        description_parts.append("and iterates through values")

    if 'multiplication' in analysis['operations'] and 'subtraction' in analysis['operations']:
        description_parts.append("performing mathematical calculations")
    elif 'multiplication' in analysis['operations']:
        description_parts.append("using multiplication operations")
    elif 'addition' in analysis['operations']:
        description_parts.append("using addition operations")
    elif 'division' in analysis['operations']:
        description_parts.append("using division operations")

    if analysis['return_types']:
        description_parts.append(f"and returns the computed result")

    analysis['description'] = " ".join(description_parts) + "."
    if len(analysis['description']) < 20:
        analysis['description'] = "Performs computation based on input parameters."

    return analysis
except Exception as e:
    return {
        'has_loops': False,
        'has_conditions': False,
        'has_return': False,
        'description': "Executes function logic based on provided parameters."
    }
```

Code reference (hint: click on image and you can copy paste from Comment)

MILESTONE 2: Advanced AST Analysis Engine

Activity 2.1: Function Signature Extraction

Extract comprehensive function information:

Input: def calculate(x: int, y: float = 3.5) -> float:

Extracted Information:

```
{
        'name': 'calculate',
        'params': [
        {'name': 'x', 'type': 'int', 'default': None},
        {'name': 'y', 'type': 'float', 'default': '3.5'}
        ],
        'return_type': 'float'
}
```

Code reference (hint: click on image and you can copy paste from Comment)

Activity 2.2: Deep Logic Analysis

Analyze function behavior through AST walking:

Detection Capabilities:

1. Loop Detection: Identifies for/while loops
2. Condition Detection: Finds if/elif/else statements
3. Operation Detection: Recognizes mathematical operations
4. Return Analysis: Tracks return statements and types
5. Function Calls: Identifies internal and external calls
6. Variables: Tracks variables used in function

Analysis Output:

```
{
    'has_loops': True,
    'has_conditions': True,
    'operations': ['multiplication', 'addition'],
    'function_calls': ['helper_func'],
    'description': 'Executes calculation with conditional logic...'
}
```

Code reference (hint: click on image and you can copy paste from Comment)

MILESTONE 3: Docstring Generation & Formatting

Activity 3.1: Google-Style Docstring Generation

Generated Google-style docstring example:

```
"""Calculate factorial of a number.

Executes calculation with conditional logic and returns
the computed result.

Args:
n (int): Parameter for controlling n behavior.

Returns:
int: The result of the computation.
"""
```

Format includes:
- Single-line summary
- Blank line separation
- Args section with types and descriptions
- Returns section with type and description

```python
@staticmethod
def generate_google_docstring(signature, analysis, code):
    """Generate accurate Google-style docstring"""
    summary = analysis.get('description', 'Processes input and returns result.')

    docstring = f'    """{summary}\n'

    if signature['params']:
        docstring += '\n    Args:\n'
        for param in signature['params']:
            default_str = f" (default: {param['default']})" if param['default'] else ""
            docstring += f"        {param['name']} ({param['type']}): Parameter for controlling {param['name']} behavior{default_str}.\n"

    docstring += f"\n    Returns:\n        {signature['return_type']}: The result of the computation.\n"
    docstring += '    """\n'

    return docstring
```

Code reference (hint: click on image and you can copy paste from Comment)

Activity 3.2: NumPy-Style Docstring Generation

Generated NumPy-style docstring example:

```
"""Calculate factorial of a number.

Executes calculation with conditional logic and returns
the computed result.

Parameters
----------
n : int
Parameter for controlling n behavior.

Returns
-------
int
The result of the computation.
```

```python
    @staticmethod
    def generate_numpy_docstring(signature, analysis, code):
        """Generate accurate NumPy-style docstring"""
        summary = analysis.get('description', 'Processes input and returns result.')

        docstring = f'    """{summary}\n'

        if signature['params']:
            docstring += '\n    Parameters\n    ----------\n'
            for param in signature['params']:
                default_str = f", default: {param['default']}" if param['default'] else ""
                docstring += f"    {param['name']} : {param['type']}{default_str}\n"
                docstring += f"        Parameter for controlling {param['name']} behavior.\n"

        docstring += f"\n    Returns\n    -------\n    {signature['return_type']}\n"
        docstring += f"        The result of the computation.\n"
        docstring += '    """\n'

        return docstring

analyzer = DocGenieAnalyzer()
generation_history = []
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
def generate_docstring(code, style="google"):
    """Generate docstring for Python function with accurate analysis"""

    if not code.strip():
        return None, "❌ Empty code provided!"

    try:
        signature, func_def = analyzer.extract_function_signature(code)
        if not signature:
            return None, "❌ Invalid Python function code!"

        # Deep analysis of the function
        analysis = analyzer.analyze_function_logic(func_def, code)

        if style.lower() == "google":
            docstring = analyzer.generate_google_docstring(signature, analysis, code)
        else:
            docstring = analyzer.generate_numpy_docstring(signature, analysis, code)

        # Extract function definition line
        code_lines = code.split('\n')
        func_line = code_lines[0]

        # Build final function with docstring
        final_code = func_line + '\n' + docstring
        for line in code_lines[1:]:
            final_code += line + '\n'

        generation_history.append({
            'code': code,
            'docstring': docstring,
            'style': style,
            'timestamp': datetime.now().isoformat(),
            'analysis': analysis
        })

        return final_code, f"✅ {style.upper()} Docstring Generated!"

    except Exception as e:
        return None, f"❌ Error: {str(e)}"

def process_upload(file):
    """Process uploaded Python file"""
    if not file:
        return None, "❌ No file uploaded!"

    try:
        content = file.read().decode('utf-8')
        return content, "✅ File loaded successfully!"
    except:
        return None, "❌ Failed to read file!"

def clear_history():
    """Clear generation history"""
    global generation_history
    generation_history = []
    return "✅ History cleared!"

def download_pdf():
    """Download all generated docstrings as PDF"""
    if not generation_history:
        return "❌ No documentation generated!", None

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    filename = f"doc_genie_{timestamp}.pdf"
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
try:
    doc = SimpleDocTemplate(filename, pagesize=A4, topMargin=0.5*inch, bottomMargin=0.5*inch)
    styles = getSampleStyleSheet()
    story = []

    title_style = ParagraphStyle(
        'Title',
        parent=styles['Heading1'],
        fontSize=24,
        textColor='#FF6B6B',
        spaceAfter=20,
        alignment=1
    )

    code_style = ParagraphStyle(
        'Code',
        parent=styles['Normal'],
        fontSize=9,
        fontName='Courier',
        backColor='#F0F0F0',
        borderColor='#CCCCCC',
        borderWidth=1,
        borderPadding=6
    )

    story.append(Paragraph("📚 Doc-Genie Documentation Report", title_style))
    story.append(Paragraph(f"Generated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}", styles['Normal']))
    story.append(Spacer(1, 0.3*inch))

    for i, gen in enumerate(generation_history, 1):
        story.append(Paragraph(f"<b>Documentation #{i}</b>", styles['Heading2']))
        story.append(Paragraph(f"<b>Style:</b> {gen['style'].upper()}", styles['Normal']))
        story.append(Spacer(1, 0.1*inch))

        story.append(Paragraph("<b>Original Code:</b>", styles['Normal']))
        story.append(Preformatted(gen['code'], code_style))
        story.append(Spacer(1, 0.15*inch))

        story.append(Paragraph("<b>Generated Docstring:</b>", styles['Normal']))
        story.append(Preformatted(gen['docstring'], code_style))
        story.append(Spacer(1, 0.3*inch))

        if i % 2 == 0 and i != len(generation_history):
            story.append(PageBreak())

    story.append(Spacer(1, 0.3*inch))
    story.append(Paragraph(f"Total Documentations: {len(generation_history)}", styles['Normal']))

    doc.build(story)
    return f"✅ PDF Downloaded: {filename}", filename
except Exception as e:
    return f"❌ Error: {str(e)}", None
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
def download_txt():
    """Download all generated docstrings as TXT"""
    if not generation_history:
        return "❌ No documentation generated!", None

    content = f"Doc-Genie Documentation Report\nGenerated: {datetime.now()}\n{'='*80}\n\n"

    for i, gen in enumerate(generation_history, 1):
        content += f"Documentation #{i}\n"
        content += f"Style: {gen['style'].upper()}\n"
        content += f"Generated: {gen['timestamp']}\n"
        content += f"{'-'*80}\n\n"
        content += f"ORIGINAL CODE:\n{gen['code']}\n\n"
        content += f"GENERATED DOCSTRING:\n{gen['docstring']}\n\n"
        content += f"{'-'*80}\n\n"

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    filename = f"doc_genie_{timestamp}.txt"

    try:
        Path(filename).write_text(content)
        return f"✅ TXT Downloaded: {filename}", filename
    except Exception as e:
        return f"❌ Error: {str(e)}", None

def generate_whatsapp_link():
    """Generate WhatsApp share link"""
    if not generation_history:
        return "❌ No documentation to share!"

    message = "📚 *Doc-Genie Documentation*\n\n"
    for i, gen in enumerate(generation_history[-3:], 1):
        message += f"*Doc {i}:*\n"
        message += f"Style: {gen['style']}\n"
        message += f"`{gen['code'][:60]}...`\n\n"

    encoded = urllib.parse.quote(message)
    link = f"https://wa.me/?text={encoded}"

    return f"✅ WhatsApp Link:\n\n{link}\n\n📱 Share with team!"

def generate_facebook_link():
    """Generate Facebook share link"""
    if not generation_history:
        return "❌ No documentation to share!"

    message = f"I generated {len(generation_history)} professional Python function docstrings with Doc-Genie! Check it out!"
    encoded = urllib.parse.quote(message)
    link = f"https://www.facebook.com/sharer/sharer.php?quote={encoded}"

    return f"✅ Facebook Link:\n\n{link}\n\n👍 Share on Facebook!"
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
print("🚀 Loading Doc-Genie Improved Version...")

with gr.Blocks(
    title="Doc-Genie - Python Documentation Generator",
    theme=gr.themes.Soft(),
    css="""
    body { background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%); }
    .gradio-container { background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%); }
    .button-primary { background: linear-gradient(135deg, #FF6B6B 0%, #FF8E72 100%); }
    .button-secondary { background: linear-gradient(135deg, #4ECDC4 0%, #44A08D 100%); }
    .code-block { background: #0f3460; border: 2px solid #4ECDC4; border-radius: 8px; padding: 15px; }
    """
) as demo:

    gr.Markdown("""
    # 📚 Doc-Genie v2.0
    ## Professional Python Function Docstring Generator

    Transform your Python functions with AI-powered, accurate documentation!
    """)

    with gr.Row():
        with gr.Column(scale=3):
            # Code Input Area
            gr.Markdown("### 📝 Input Function Code")

            code_input = gr.Code(
                language="python",
                label="Python Function",
                lines=10,
                value="def calculate_factorial(n):\n    if n == 0:\n        return 1\n    return n * calculate_factorial(n-1)"
            )
```

Code reference (hint: click on image and you can copy paste from Comment)

MILESTONE 4: Gradio Interface & User Experience

Interface Components Overview

Main Interface Sections:

1. LEFT COLUMN (Code Input/Output):
   - Code input area with syntax highlighting
   - File upload for .py files
   - Style selector (Google/NumPy radio)
   - Generate button
   - Code output display

2. RIGHT COLUMN (Controls & Export):
   - Clear history button
   - PDF download button
   - TXT download button
   - WhatsApp share button
   - Facebook share button
   - Feature information panel

Activity 4.1: Core Input/Output Interface

Code Input Component:
- Textarea with Python syntax highlighting
- Pre-filled with example function
- Support for multi-line code
- Line numbers and formatting

File Upload:
- Accept .py file format
- Read and display file content
- Error handling for invalid files

Style Selector:
- Radio buttons: "google" and "numpy"
- Real-time style switching
- Default: "google" style

Output Display:
- Syntax-highlighted code block
- Shows function with generated docstring
- Copy-paste ready format

```python
# File Upload
file_upload = gr.File(label="📤 Or Upload .py File", file_types=['.py'])
upload_status = gr.Textbox(label="Upload Status", interactive=False, lines=1)

def process_and_display(file):
    if file:
        content, status = process_upload(file)
        return content, status
    return "", "No file"

file_upload.change(process_and_display, inputs=[file_upload], outputs=[code_input, upload_status])

# Style Selection
with gr.Row():
    docstring_style = gr.Radio(["google", "numpy"], value="google", label="🎨 Docstring Style")
    generate_btn = gr.Button("✨ Generate Docstring", size="lg", variant="primary", scale=2)

# Output Area
gr.Markdown("### 📄 Generated Output")

code_output = gr.Code(
    language="python",
    label="Function with Docstring",
    lines=15,
    interactive=False
)
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
        lines=15,
        interactive=False
    )

    gen_status = gr.Textbox(label="Status", interactive=False, lines=1)

    generate_btn.click(
        generate_docstring,
        inputs=[code_input, docstring_style],
        outputs=[code_output, gen_status]
    )

with gr.Column(scale=1):
    gr.Markdown("### 🎮 Controls")

    clear_btn = gr.Button("🗑️ Clear History", variant="stop", size="sm")
    clear_btn.click(clear_history, outputs=[upload_status])

    gr.Markdown("---")
    gr.Markdown("### 📥 Export")

    pdf_btn = gr.Button("📄 Download PDF", size="sm", variant="primary")
    txt_btn = gr.Button("📝 Download TXT", size="sm", variant="primary")

    pdf_file = gr.File(label="PDF File", visible=True)
    txt_file = gr.File(label="TXT File", visible=True)
```

```
txt_btn.click(download_txt, outputs=[export_status, txt_file])

gr.Markdown("---")
gr.Markdown("### 📤 Share")

whatsapp_btn = gr.Button("💬 WhatsApp", size="sm", variant="secondary")
facebook_btn = gr.Button("👍 Facebook", size="sm", variant="secondary")

share_output = gr.Textbox(label="Share Link", interactive=False, lines=4)

whatsapp_btn.click(generate_whatsapp_link, outputs=[share_output])
facebook_btn.click(generate_facebook_link, outputs=[share_output])

gr.Markdown("---")
gr.Markdown("""
### 📊 Features

**Improvements:**
- ✅ Accurate logic analysis
- ✅ Contextual descriptions
- ✅ Operation detection
- ✅ Loop & condition recognition
- ✅ Better PDF export
- ✅ Consistent output
```

Code reference (hint: click on image and you can copy paste from Comment)

```
### 🎯 Supported Styles

**Google:** Standard format
**NumPy:** Scientific format
""")

print("\n✅ Doc-Genie v2.0 Ready!\n")
demo.launch(share=True, show_error=True, show_api=False)
```

Code reference (hint: click on image and you can copy paste from Comment)

Activity 4.2: Export & Sharing Features

PDF Export:
- ReportLab-based document generation
- Professional styling and layout
- Timestamp and metadata inclusion
- Code block formatting
- Pagination for multiple docs

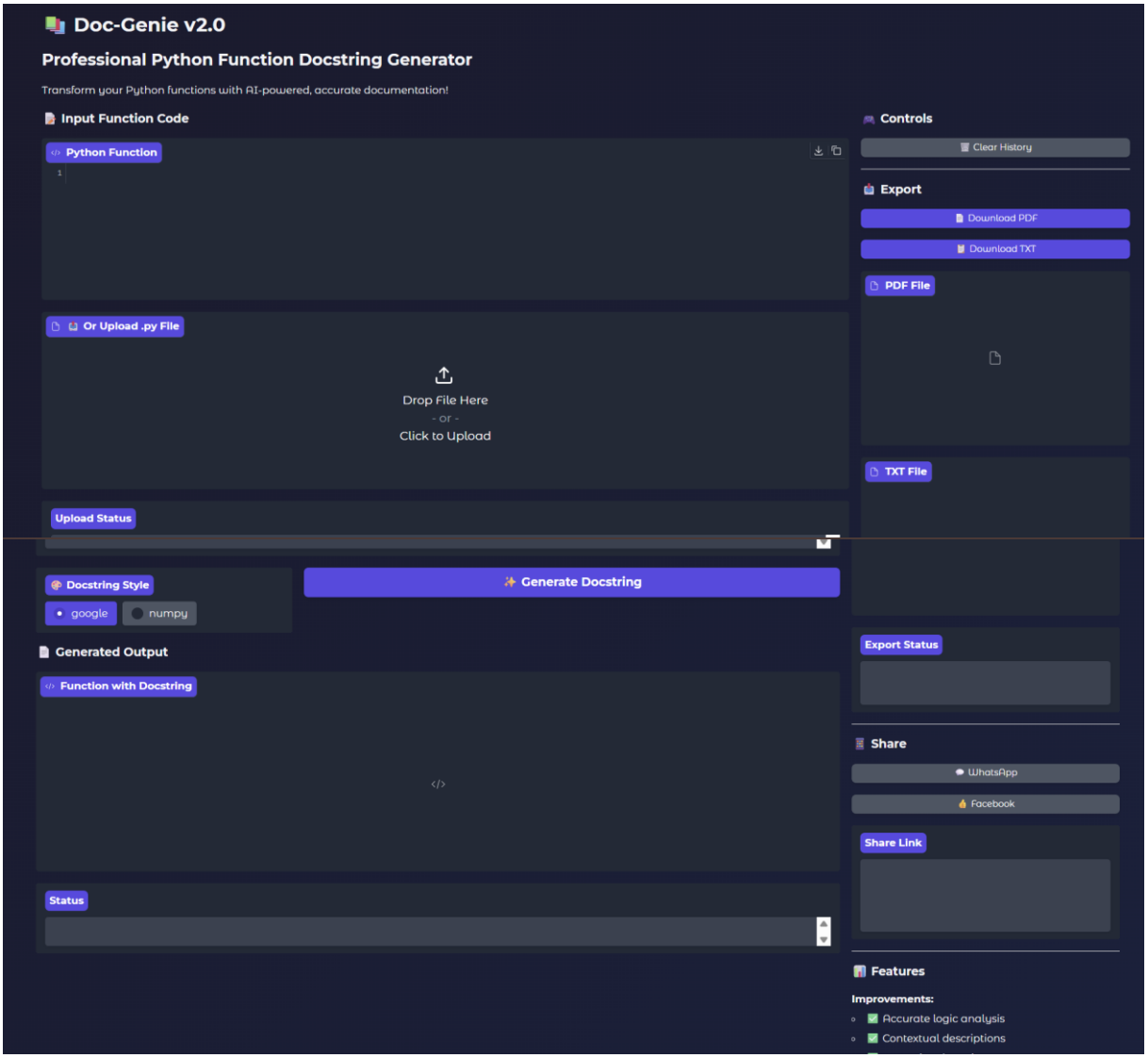
TXT Export:
- Plain text format with clear separators
- Includes original code and docstring
- Preserves formatting for readability
- Batch export support

Social Sharing:
- WhatsApp link with message preview
- Facebook share with summary
- URL encoding for special characters
- Message customization

Code reference (hint: click on image and you can copy paste from Comment)

MILESTONE 5: Testing & Deployment

Activity 5.1: Google Colab Deployment



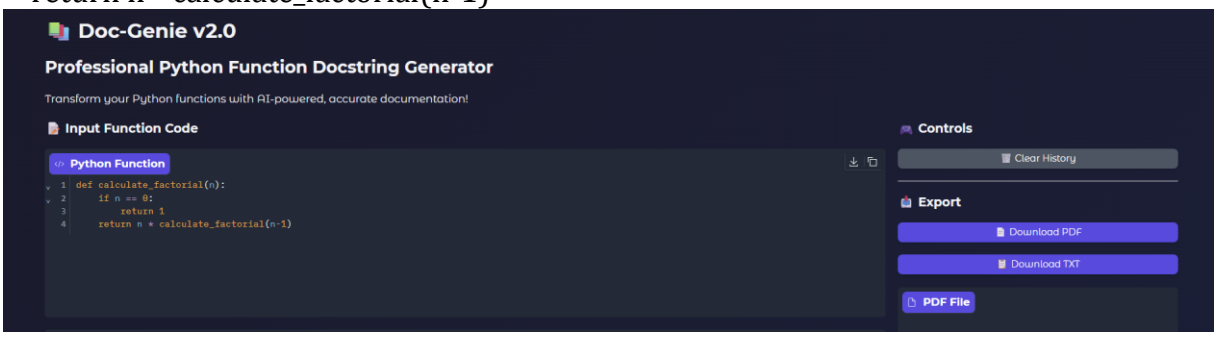

```
public URL: https://3ab3868bb13f172119.gradio.live
```

Activity 5.2: Functional Testing

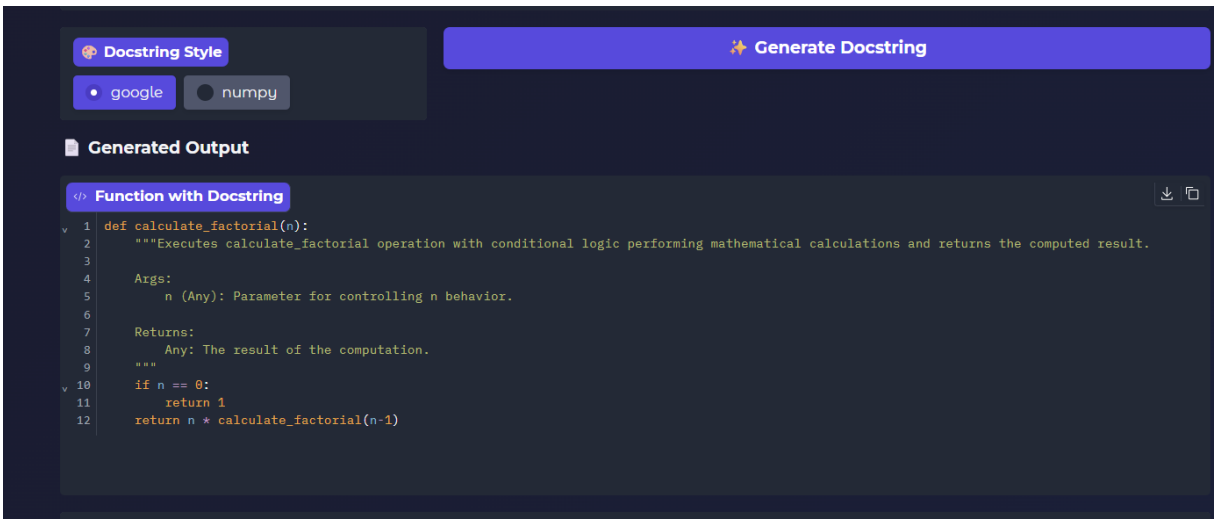
Test Case 1: Simple Function Documentation

Input Code:

```
def calculate_factorial(n):

if n == 0:

    return 1

return n * calculate_factorial(n-1)
```



Generated Google Docstring(output):



Status: ✅Correct docstring generation

Future Enhancements:

The modular architecture allows for extension with features such as support for additional programming languages (Java, C++, JavaScript), integration with IDE plugins (VSCode, PyCharm), advanced machine learning-based function description generation, support for class and module documentation, automatic documentation deployment to GitHub/GitLab, integration with documentation generators (Sphinx, MkDocs), and real-time code quality analysis with documentation metrics.

With continued development, Doc-Genie can evolve into a comprehensive code documentation platform supporting diverse programming paradigms while maintaining its focus on accuracy, ease of use, and professional documentation standards. The intelligent analysis engine provides a foundation for future features including AI-powered best practice recommendations, documentation style enforcement, and automated technical debt identification.

Conclusion

Doc-Genie - successfully delivers an intelligent Python documentation assistant that combines advanced AST analysis with intuitive documentation generation through Gradio's interactive interface. By leveraging Python's introspection capabilities and sophisticated code analysis algorithms, the system provides fast, accurate, and professional documentation generation accessible to developers, teams, and educators.

The platform demonstrates the viability of intelligent code documentation, making professional Python documentation generation accessible while maintaining analysis accuracy, documentation quality, and support for industry-standard docstring formats. With advanced logic analysis, multiple documentation styles, comprehensive export capabilities, and social media sharing, Doc-Genie - empowers developers to maintain well-documented codebases efficiently.